



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

PowerShell Scripting and Automation

As incredible as PowerShell is, it's only as good as how we use it. Much of the PowerShell language and functionality lends itself to being utilized in an automated fashion. Having the ability to build scripts and modules for us in PowerShell (no matter how simple or complex) can ease our administrative burden or clear some easy tasks off our plate as pentesters. This module will discuss the pieces and parts that make up a PowerShell script and module. By the end, we will have created our own easy-to-use and customizable module.

Understanding PowerShell Scripting

PowerShell, by its nature, is modular and allows for a significant amount of control with its use. The traditional thought when dealing with scripting is that we are writing some form of an executable that performs tasks for us in the language it was created. With PowerShell, this is still true, with the exception that it can handle input from several different languages and file types and can handle many different object types. We can utilize singular scripts in the usual manner by calling them utilizing `.\script` syntax and importing modules using the `Import-Module` cmdlet. Now let's talk a bit about scripts and modules.

Scripts vs. Modules

The easiest way to think of it is that a script is an executable text file containing PowerShell cmdlets and functions, while a module can be just a simple script, or a collection of multiple script files, manifests, and functions bundled together. The other main difference is in their use. You would typically call a script by executing it directly, while you can import a module and all of the associated scripts and functions to call at your whim. For the sake of this section, we will discuss them using the same term, and everything we talk about in a module file works in a standard PowerShell script. First up is `file extensions` and what they mean to us.

File Extensions

To familiarize ourselves with some file extensions we will encounter while working with PowerShell scripts and modules, we have put together a small table with the extensions and their descriptions.

PowerShell Extensions

Extension	Description
ps1	The <code>*.ps1</code> file extension represents executable PowerShell scripts.
psm1	The <code>*.psm1</code> file extension represents a PowerShell module file. It defines what the module is and what is contained within it.
psd1	The <code>*.psd1</code> is a PowerShell data file detailing the contents of a PowerShell module in a table of key/value pairs.

These are the main extensions we are concerned with right now. In reality, PowerShell modules can have many different accompanying files with various extensions, but they are not requirements for what we are trying to do. If you wish for a deeper dive into PowerShell script files, and help files, check out this [Post](#)



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

create them. This process is simple. It just takes a bit of prior planning. Consider this scenario:

Scenario: We have found ourselves performing the same checks over and over when administering hosts. So to expedite our tasks, we will create a PowerShell module to run the checks for us and then output the information we ask for. Our module, when used, should output the host's `computer name`, `IP address`, and basic `domain information`, and provide us with the output of the `C:\Users\` directory so we can see what users have interactively logged into that host.

Now that we know what's going into our module, it's time to start building it out.

Module Components

A module is made up of `four` essential components:

1. A `directory` containing all the required files and content, saved somewhere within `$env:PSModulePath`.
 - This is done so that when you attempt to import it into your PowerShell session or Profile, it can be automatically found instead of having to specify where it is.
2. A `manifest` file listing all files and pertinent information about the module and its function.
 - This could include associated scripts, dependencies, the author, example usage, etc.
3. Some code file - usually either a PowerShell script (`.ps1`) or a (`.psm1`) module file that contains our script functions and other information.
4. Other resources the module needs, such as help files, scripts, and other supporting documents.

This setup is standard practice but not strictly necessary. We could have our module be just a `*.psm1` file that contains our scripts and context, skipping the manifest and other helper files. PowerShell would be able to interpret and understand what to do in either instance. For the sake of propriety, we will work on building out a standard PowerShell module, including the manifest file and some built-in help functionality.

Making a Directory to Hold Our Module

Making a directory is super simple, as discussed in earlier sections. Before we go any further, we need to create the directory to hold our module. This directory should be in one of the paths within `$env:PSModulePath`. If unsure as to what those paths are, you can call the variable to see where the best place would be. So we are going to make a folder named `quick-recon`.

Mkdir

PowerShell Scripting and Automation				
PS C:\htb> mkdir quick-recon				
Directory: C:\Users\MTanaka\Documents\WindowsPowerShell\Modules				
Mode	LastWriteTime		Length	Name
----	-----		-----	----
d-----	10/31/2022	7:38 AM		quick-recon

Now that we have our directory, we can create the module. Let's discuss a `module manifest` file for a second.



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

A module manifest is a simple `.psd1` file that contains a hash table. The keys and values in the hash table perform the following functions:

- Describe the `contents` and `attributes` of the module.
- Define the `prerequisites`. (specific modules from outside the module itself, variables, functions, etc.)
- Determine how the `components` are `processed`.

If you add a manifest file to the module folder, you can reference multiple files as a single unit by referencing the manifest. The `manifest` describes the following information:

- `Metadata` about the module, such as the module version number, the author, and the description.
- `Prerequisites` needed to import the module, such as the Windows PowerShell version, the common language runtime (CLR) version, and the required modules.
- `Processing` directives, such as the scripts, formats, and types to process.
- `Restrictions` on the module members to export, such as the aliases, functions, variables, and cmdlets to export.

We can quickly create a manifest file by utilizing `New-ModuleManifest` and specifying where we want it placed.

New-ModuleManifest

PowerShell Scripting and Automation

```
PS C:\htb> New-ModuleManifest -Path C:\Users\MTanaka\Documents\WindowsPowerShell\Modules\quick-recon\quick-recon.manifest

# Module manifest for module 'quick-recon'
#
# Generated by: MTanaka
#
# Generated on: 10/31/2022
#

@{

# Script module or binary module file associated with this manifest.
# RootModule = ''

# Version number of this module.
ModuleVersion = '1.0'

<SNIP>
```

By issuing the command above, we have provisioned a `new` manifest file populated with the default considerations. The `-PassThru` modifier lets us see what is being printed in the file and on the console. We can now go in and fill in the sections we want with the relevant info. Remember that all the lines in the manifest files are optional except for the `ModuleVersion` line. Editing the manifest will be easiest done from a GUI where you can utilize a text editor or IDE such as VSCode. If we were to complete our manifest file now for this module, it would appear something like this:

Sample Manifest

Code: `powershell`

```
# Module manifest for module 'quick-recon'
```



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

```
# Generated on 05/05/2022
#

@{

# Script module or binary module file associated with this manifest.
# RootModule = 'C:\Users\MTanaka\WindowsPowerShell\Modules\quick-recon\quick-recon.psm1'

# Version number of this module.
ModuleVersion = '1.0'

# ID used to uniquely identify this module
GUID = '0a062bb1-8a1b-4bdb-86ed-5adbe1071d2f'

# Author of this module
Author = 'MTanaka'

# Company or vendor of this module
CompanyName = 'Greenhorn.Corp.'

# Copyright statement for this module
Copyright = '(c) 2022 Greenhorn.Corp. All rights reserved.'

# Description of the functionality provided by this module
Description = 'This module will perform several quick checks against the host for Reconnaissance of key information'

# Functions to export from this module, for best performance, do not use wildcards and do not delete the entry, use the @() notation
FunctionsToExport = @()

# Cmdlets to export from this module, for best performance, do not use wildcards and do not delete the entry, use the @() notation
CmdletsToExport = @()

# Variables to export from this module
VariablesToExport = '*'

# Aliases to export from this module, for best performance, do not use wildcards and do not delete the entry, use the @() notation
AliasesToExport = @()

# List of all modules packaged with this module
# ModuleList = @()

# List of all files packaged with this module
# FileList = @()
}
```

We can come back to the manifest later and add in the **functions**, **cmdlets**, and **variables** we want to allow for export. We need to build and finish the script first.

Create Our Script File

We can use the **New-Item** (ni) cmdlet to create our file.

New-Item



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	10/31/2022 9:07 AM	0	quick-recon.psm1

Easy enough, right? Now to fill in this beast.

Importing Modules You Need

If our new PowerShell requires other modules or cmdlets from within them to operate correctly, we will place an `Import-Module` string at the beginning of our script file. The use of `Import-Module` in this manner functions much like it would if we issued it from within the shell; it calls and loads the modules we need before executing our script. To accomplish the goals we have for this module, many of the cmdlets and functions are already built-in into PowerShell. We do need one from the ActiveDirectory PowerShell module, however. So let's add an import line for the `ActiveDirectory` module.

Import Into Our Module

```
Code: powershell

Import-Module ActiveDirectory
```

Pretty simple right? Now we have our module script file `quick-recon.psm1`, and we have added an `import-module` statement within. Now we can get to the meat of the file, our `functions`.

Functions and doing work with Powershell

We need to do four main things with this module:

- Retrieve the host ComputerName
- Retrieve the hosts IP configuration
- Retrieve basic domain information
- Retrieve an output of the "C:\Users" directory

To get started, let's focus on the ComputerName output. We can get this many ways with various cmdlets, modules, and DOS commands. Our script will utilize the environment variable (`$env:ComputerName`) to acquire the hostname for the output. To make our output easier to read later, we will use another variable named `$hostname` to store the output from the environment variable. To capture the IP address for the active host adapters, we will use `IPConfig` and store that info in the variable `$IP`. For Basic domain information, we will use `Get-ADDomain` and store the output into `$Domain`. Lastly, we will grab a listing of the user folders in C:\Users\ with `Get-ChildItem` and store it in `$Users`. To create our variables, we must first specify a name like (`$Hostname`), append the "=" symbol, and then follow it with the action or values we want it to hold. For example, the first variable we need, `$Hostname`, would appear like so: (`$Hostname = $env:ComputerName`). Now let's dive in and create the rest of our variables for use.

Variables

```
Code: powershell

Import-Module ActiveDirectory
```



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

```
$Users = Get-ChildItem C:\Users\
```

Our variables are now set to run singular commands or functions, grabbing the needed output. Now let's format that data and give ourselves some nice output. We can do this by writing the result to a file using `New-Item` and `Add-Content`. To make things easier, we will make this output process into a callable function called `Get-Recon`.

Output Our Info

Code: powershell

```
Import-Module ActiveDirectory

function Get-Recon {

    $Hostname = $env:ComputerName

    $IP = ipconfig

    $Domain = Get-ADDomain

    $Users = Get-ChildItem C:\Users\

    new-Item ~\Desktop\recon.txt -ItemType File

    $Vars = "***---Hostname info---***", $Hostname, "***---Domain Info---***", $Domain, "*****IP INFO---***",

    Add-Content ~\Desktop\recon.txt $Vars

}
```

`New-Item` creates our output file for us first, then notice how we utilized one more variable (`$Vars`) to format our output. We call each variable and insert a descriptive line in between each. Lastly, the `Add-Content` cmdlet appends the data we gather into a file called `recon.txt` by writing the results of `$Vars`. Our function is shaping up now. Next, we need to add some comments to our file so that others can understand what we are trying to accomplish and why we did it the way we did.

Comments within the Script

The `(#)` will tell PowerShell that the line contains a comment within your script or module file. If your comments are going to encompass several lines, you can use the `<#` and `#>` to wrap several lines as one large comment like seen below:

Comment Blocks

Code: powershell

```
# This is a single-line comment.

<# This line and the following lines are all wrapped in the Comment specifier.
Nothing with this window will be ready by the script as part of a function.
This text exists purely for the creator and us to convey pertinent information.

#>
```



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

Code: powershell

```
Import-Module ActiveDirectory

function Get-Recon {
    # Collect the hostname of our PC.
    $Hostname = $env:ComputerName
    # Collect the IP configuration.
    $IP = ipconfig
    # Collect basic domain information.
    $Domain = Get-ADDomain
    # Output the users who have logged in and built out a basic directory structure in "C:\Users\".
    $Users = Get-ChildItem C:\Users\
    # Create a new file to place our recon results in.
    new-Item ~\Desktop\recon.txt -ItemType File
    # A variable to hold the results of our other variables.
    $Vars = "***---Hostname info---***", $Hostname, "***---Domain Info---***", $Domain, "*****IP INFO---***",
    # It does the thing
    Add-Content ~\Desktop\recon.txt $Vars
}
```

It's as simple as that. Nothing too crazy with comments. Now we need to include a bit of `help` syntax so others can understand how to use our module.

Including Help

PowerShell utilizes a form of `Comment-based help` to embed whatever you need for the script or module. We can utilize `comment blocks` like those we discussed above, along with recognized `keywords` to build the help section out and even call it using `Get-Help` afterward. When it comes to placement, we have `two` options here. We can place the help within the function itself or outside of the function in the script. If we wish to place it within the function, it must be at the beginning of the function, right after the opening line for the function, or at the end of the function, one line after the last action of the function. If we place it within the script but outside of the function itself, we must place it above our function with no more than one line between the help and function. For a deeper dive into help within PowerShell, check out this [article](#). Now let's define our help section. We will place it outside of the function at the top of the script for now.

Module Help

Code: powershell

```
Import-Module ActiveDirectory

<#
.Description
This function performs some simple recon tasks for the user. We import the module and issue the 'Get-Recon' command.

.Example
After importing the module run "Get-Recon"
'Get-Recon

Directory: C:\Users\MTanaka\Desktop
```



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

```
.Notes
Remote Recon functions coming soon! This script serves as our initial introduction to writing functions and sc

#>

function Get-Recon {
<SNIP>
```

Notice our use of **keywords**. To specify a keyword within the comment block, we use the syntax `.<keyword>` and then place the flavor text underneath. We only specified **Description**, **Example**, and **Notes**, but several more keywords can be placed in the help block. To see all the available keywords, reference this article on [Comment Based Help Keywords](#). Our last portion to discuss before wrapping everything up into our nice PowerShell Module file, is Exporting and Protecting functions.

Protecting Functions

We may add functions to our scripts that we do not want to be accessed, exported, or utilized by other scripts or processes within PowerShell. To protect a function from being exported or to explicitly set it for export, the **Export-ModuleMember** is the cmdlet for the job. The contents are exportable if we leave this out of our script modules. If we place it in the file but leave it blank like so:

Exclude From Export

```
Code: powershell

Export-ModuleMember
```

It ensures that the module's variables, aliases, and functions cannot be **exported**. If we wish to specify what to export, we can add them to the command string like so:

Export Specific Functions and Variables

```
Code: powershell

Export-ModuleMember -Function Get-Recon -Variable Hostname
```

Alternatively, if you only wanted to export all functions and a specific variable, for example, you could issue the `*` after `-Function` and then specify the Variables to export explicitly. So let's add the **Export-ModuleMember** cmdlet to our script and specify that we want to allow our function **Get-Recon** and our variable **Hostname** to be available for export.

Export Line Addition

```
Code: powershell

<SNIP>
function Get-Recon {
    # Collect the hostname of our PC
    $Hostname = $env:ComputerName
    # Collect the IP configuration
    $IP = ipconfig
```




To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

```
## Create a new file to place our recon results in
new-Item ~\Desktop\recon.txt -ItemType File
# A variable to hold the results of our other variables
$Vars = "***---Hostname info---***", $Hostname, "***---Domain Info---***", $Domain, "***---IP INFO---***",
# It does the thing
Add-Content ~\Desktop\recon.txt $Vars
}

Export-ModuleMember -Function Get-Recon -Variable Hostname
```

Scope

When dealing with scripts, the PowerShell session, and how stuff is recognized at the Commandline, the concept of Scope comes into play. Scope, in essence, is how PowerShell recognizes and protects objects within the session from unauthorized access or modification. PowerShell currently uses **three** different Scope levels:

Scope Levels

Scope	Description
Global	This is the default scope level for PowerShell. It affects all objects that exist when PowerShell starts, or a new session is opened. Any variables, aliases, functions, and anything you specify in your PowerShell profile will be created in the Global scope.
Local	This is the current scope you are operating in. This could be any of the default scopes or child scopes that are made.
Script	This is a temporary scope that applies to any scripts being run. It only applies to the script and its contents. Other scripts and anything outside of it will not know it exists. To the script, its scope is the local scope.

This matters to us if we do not want anything outside the scope we are running the script in to access its contents. Additionally, we can have child scopes created within the main scopes. For example, when you run a script, the script scope is instantiated, and then any function that is called can also spawn a child scope surrounding that function and its included variables. If we wanted to ensure that the contents of that specific function were not accessible to the rest of the script or the PowerShell session itself, we could modify its scope. This is a complex topic and something above the level of this module currently, but we felt it was worth mentioning. For more on Scope in PowerShell, check out the documentation [here](#).

Putting It All Together

Now that we have gone through and created our pieces and parts let's see it all together.

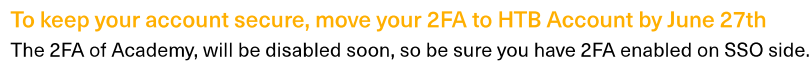
Final Product

Code: **powershell**

```
import-module ActiveDirectory

<#
.Description
This function performs some simple recon tasks for the user. We import the module and then issue the 'Get-Recon'

.Example
After importing the module run "Get-Recon"
'Get-Recon
```



Enable now

```

Mode                LastWriteTime         Length Name
-----
-a-----          11/3/2022  12:46 PM              0 recon.txt '

.Notes
Remote Recon functions coming soon! This script serves as our initial introduction to writing functions and sc

#>
function Get-Recon {
    # Collect the hostname of our PC
    $Hostname = $env:ComputerName
    # Collect the IP configuration
    $IP = ipconfig
    # Collect basic domain information
    $Domain = Get-ADDomain
    # Output the users who have logged in and built out a basic directory structure in "C:\Users"
    $Users = Get-ChildItem C:\Users\
    # Create a new file to place our recon results in
    new-Item ~\Desktop\recon.txt -ItemType File
    # A variable to hold the results of our other variables
    $Vars = "****--Hostname info---****", $Hostname, "****--Domain Info---****", $Domain, "****--IP INFO---****",
    # It does the thing
    Add-Content ~\Desktop\recon.txt $Vars
}

Export-ModuleMember -Function Get-Recon -Variable Hostname

```

And there we have it, our full module file. Our use of Comment-based help, functions, variables and content protection makes for a dynamic and clear-to-read script. From here we can save this file in our Module directory we created and import it from within PowerShell for use.

Importing the Module For Use

PowerShell Scripting and Automation

```
PS C:\htb> Import-Module 'C:\Users\MTanaka\Documents\WindowsPowerShell\Modules\quick-recon.psm1'
```

```
PS C:\Users\MTanaka\Documents\WindowsPowerShell\Modules\quick-recon> get-module
```

ModuleType	Version	Name	ExportedCommands
Manifest	3.1.0.0	Microsoft.PowerShell.Management	{Add-Computer, Add-Content, Checkpoint-Computer, Clea
Script	2.0.0	PSReadline	{Get-PSReadLineKeyHandler, Get-PSReadLineOption, Rem
Script	0.0	quick-recon	Get-Recon

Perfect. We can see that our module was imported using the `Import-Module` cmdlet, and to ensure it was loaded into our session, we ran the `Get-Module` cmdlet. It has shown us that our module `quick-recon` was imported and has the command `Get-Recon` that could be exported. We can also test the Comment-based help by trying to run `Get-Help` against our module.

Help Validation

PowerShell Scripting and Automation



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

SYNOPSIS

SYNTAX

```
Get-Recon [<CommonParameters>]
```

DESCRIPTION

This function performs some simple recon tasks for the user. We simply import the module and then issue the 'Get-Recon' command to retrieve our output. Each variable and line within the function and script are commented for your understanding. Right now, this only works on the local host from which you run it, and the output will be saved to a file named 'recon.txt' on the Desktop of the user who opened the shell. Remote Recon functions coming soon.

RELATED LINKS

REMARKS

To see the examples, type: "get-help Get-Recon -examples."
For more information, type: "get-help Get-Recon -detailed."
For technical information, type: "get-help Get-Recon -full."

Our help works as well. So we now have a fully functioning module for our use. We can use this as a basis for anything we build further and could even modify this one to encompass more reconnaissance functions in the future.

This was a simple example of what can be done from an automation perspective with PowerShell, but a great way to see it built and in use. We can use module building and scripting to our advantage and simplify our processes as we go. Saving time ultimately enables us to do more as operators and spend time on other tasks that need our attention. If you would like a copy of the quick-recon module for your use, there is a copy saved in the [Resources](#) of this module at the top right corner of any section page.

VPN Servers

 Warning: Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

PROTOCOL

 UDP 1337  TCP 443

[DOWNLOAD VPN CONNECTION FILE](#)



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

39ms ▼

Terminate Pwnbox to switch location



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

Enable now

Start Instance

∞ / 1 spawns left

Waiting to start...

← Previous

Next →

+10 Streak pts

✓ Mark Complete & Next

 Cheat Sheet

 Resources

Table of Contents

Introduction



CMD

 Command Prompt Basics



 Getting Help



 System Navigation



 Working with Directories and Files



 Gathering System Information



 Finding Files and Directories



 Environment Variables



 Managing Services



 Working With Scheduled Tasks



PowerShell

 CMD Vs. PowerShell



 All About Cmdlets and Modules



 User and Group Management



 Working with Files and Directories





To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

-  Working with the Registry 
-  Working with the Windows Event Log 
-  Networking Management from The CLI 
-  Interacting with The Web 
-  [PowerShell Scripting and Automation](#)

Finish Strong

-  Skills Assessment

Beyond This Module

My Workstation

O F F L I N E

 Start Instance

 / 1 spawns left